

Week 3 cheat sheet: pandas and describing data

Quantitative Methods in Neuroscience, 2026-27. Keep it next to you while you work.

Opening a table

```
trials = pd.read_csv(ROOT / "data" / "ibl_2afc.csv.gz")
trials.head()      # the first five rows
trials.shape       # (rows, columns)
trials.info()      # names, types, how many are filled
trials.dtypes      # what kind of values each holds
trials["rt_s"].describe()
```

Columns

```
trials["rt_s"]      # a Series, one column
trials[["rt_s", "correct"]] # a smaller DataFrame
trials["fast"] = trials["rt_s"] < 0.5 # a new one
```

The double brackets are a list of names, not a typo.

Rows

```
trials.iloc[0]      # by position, like a list
trials.loc[0, "rt_s"] # by label, the index value
```

The two coincide on a freshly loaded table and stop coinciding the moment you filter.

Filtering

```
fast = trials[trials["rt_s"] < 0.5]
both = trials[(trials["rt_s"] < 0.5)
              & (trials["correct"] == 1)]
```

& and, with brackets around each condition
| or
~ not

Missing values

```
trials["rt_s"].isna().sum() # how many are missing
trials.dropna(subset=["rt_s"])
```

Most calculations skip them silently, so count them before trusting an average.

Joining two tables

```
merged = trials.merge(mice, on="subject_id",
                      how="left")
```

One row per trial, with the columns of the mouse attached to each of its trials.

Grouping

```
trials.groupby("cohort")["correct"].mean()

trials.groupby("subject_id").agg(
    accuracy=("correct", "mean"),
    n=("correct", "count"),
).reset_index()
```

Split into groups, compute per group, put the result back together.

Random numbers

```
rng = np.random.default_rng(0) # 0 is the seed
rng.normal(loc=0, scale=1, size=100)
rng.uniform(0, 1, size=100)
rng.binomial(n=1, p=0.7, size=100)
rng.choice(values, size=10, replace=True)
```

The same seed gives the same numbers, which is what makes a result reproducible.

Centre and spread

x.mean() pulled by outliers and by a long tail
x.median() the middle value, barely moved by them
np.std(x, ddof=1) spread of the data, the n - 1 version
np.percentile(x, [25, 75]) the quartiles

Always ddof=1 on a sample.

SD against SEM

```
sem = np.std(x, ddof=1) / np.sqrt(len(x))
```

The **SD** says how spread out the observations are and does not shrink with more data. The **SEM** says how precisely you know the mean and shrinks as the square root of n. Error bars mean nothing until you say which one they are.

Distributions in one line

```
sns.histplot(data=df, x="rt_s", ax=ax)
sns.violinplot(data=df, x="cohort", y="rt_s", ax=ax)
sns.boxplot(data=df, x="cohort", y="rt_s", ax=ax)
sns.stripplot(data=df, x="cohort", y="rt_s", ax=ax)
```

Everything seaborn draws is a Matplotlib figure, so ax.set_xlabel still works.

The two results of the week

Law of large numbers. As n grows, the sample mean settles onto the true mean.

Central limit theorem. Whatever the shape of the data, the distribution of the *mean* over repeated samples becomes Normal, with spread $SD/\sqrt{n}$.